Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЁТ
к лабораторной работе №3
на тему
«РАБОТА СО СХЕМАМИ ДАННЫХ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Написать функцию, на вход которой подаются два текстовых параметра (*dev_schema_name*, *prod_schema_name*), являющихся названиями схем баз данных. На выход предоставить перечень таблиц, которые есть в схеме Dev, но нет в Prod, либо в которых различается структура таблиц. Наименования таблиц должны быть отсортированы в соответствии с очередностью их возможного создания в схеме Prod (необходимо учитывать *FOREIGN KEY* в схеме). В случае закольцованных связей выводить соответствующее сообщение.

2 Доработать предыдущий скрипт с учетом возможности сравнения не только таблиц, но и процедур, функций, индексов, пакетов.

3 Доработать предыдущий скрипт с генерацией *DDL*-скрипта на обновление объектов, а также с учетом необходимости удаления в схеме Prod объектов, отсутствующих в схеме Dev.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Схемы данных в Oracle

В СУБД *Oracle* схема представляет собой логическую совокупность объектов базы данных (таблиц, представлений, индексов, процедур и т.д.), принадлежащих конкретному пользователю. Каждый пользователь имеет собственную схему, имя которой совпадает с именем пользователя. Для обращения к объектам чужой схемы используется синтаксис *schema_name.object_name*.

2.2 Словарные представления Oracle

Для получения метаданных об объектах базы данных используются системные представления (*data dictionary views*):

– *ALL_TABLES* — содержит информацию о таблицах, доступных текущему пользователю;

– *ALL_TAB_COLUMNS* — описывает колонки всех таблиц с указанием типа данных, длины и признака *NULL*

– *ALL_CONSTRAINTS* — содержит информацию об ограничениях целостности, включая внешние ключи (*constraint_type = 'R'*);

– *ALL_INDEXES* — описывает индексы, созданные в базе данных;

– *ALL_OBJECTS* — содержит информацию обо всех объектах базы данных.

2.3 Топологическая сортировка и обход в глубину

При определении порядка создания таблиц необходимо учитывать зависимости через внешние ключи (*FOREIGN KEY*). Таблица, на которую ссылаются, должна быть создана ранее ссылающейся таблицы.

Для определения порядка используется алгоритм топологической сортировки на основе обхода в глубину (*Depth-First Search*). Вершины графа представляют таблицы, а рёбра — зависимости через внешние ключи. Алгоритм использует три состояния вершины:

– *Белая* (0) — вершина не посещена;

– *Серая* (1) — вершина в текущем стеке рекурсии;

– *Чёрная* (2) — вершина полностью обработана.

Попадание в серую вершину при обходе означает наличие циклической зависимости.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Создание типов данных

Для возврата структурированного результата из функции были созданы пользовательские типы: *t_schema_comp_row* для одной строки результата и *t_schema_comp_table* для коллекции таких строк.

Листинг 3.1 – Определение типов для возврата результата

```
CREATE OR REPLACE TYPE t_schema_comp_row AS OBJECT (
    table_name  VARCHAR2(128),
    status      VARCHAR2(50),
    details     CLOB,
    sort_order  NUMBER
);

CREATE OR REPLACE TYPE t_schema_comp_table AS TABLE OF
t_schema_comp_row;
```

3.2 Создание схем DEV и PROD

Для тестирования функции сравнения были созданы две схемы: *DEV_USER* (разработка) и *PROD_USER* (промышленная). Схемы содержат идентичные таблицы *DEPARTMENTS*, *STUDENTS*, *COURSES* и *ENROLLMENTS* с внешними ключами.

Для имитации различий в схеме *DEV_USER* были добавлены:

– колонка *EMAIL* в таблицу *STUDENTS*

– колонка *GRADE* в таблицу *ENROLLMENTS*

– таблица *INSTRUCTORS*, отсутствующая в *PROD_USER*

– индексы *IDX_STUDENTS_NAME* и *IDX_STUDENTS_EMAIL*, отсутствующие в *PROD_USER*

– процедуры *GET_STUDENT_COUNT* и *ADD_STUDENT*, отсутствующие в *PROD_USER*

– функции *GET_DEPT_NAME* и *GET_STUDENT_EMAIL*, отсутствующие в *PROD_USER*

– пакет *UNIVERSITY_PKG*, отсутствующий в *PROD_USER*.

В схеме *PROD_USER* были созданы устаревшие объекты для тестирования удаления:

– таблица *OLD_TABLE*

– индекс *IDX_OLD*.

Листинг 3.2 – Создание таблиц в схеме DEV_USER

```
CREATE TABLE DEPARTMENTS (
    DEPT_ID   NUMBER(10) PRIMARY KEY,
    DEPT_NAME VARCHAR2(100) NOT NULL,
```

```
    BUILDING  VARCHAR2(50)
);

CREATE TABLE STUDENTS (
    STUDENT_ID   NUMBER(10) PRIMARY KEY,
    FIRST_NAME   VARCHAR2(50) NOT NULL,
    LAST_NAME    VARCHAR2(50) NOT NULL,
    EMAIL        VARCHAR2(100),  -- отсутствует в PROD
    DEPT_ID      NUMBER(10),
    CONSTRAINT FK_STUDENTS_DEPT FOREIGN KEY (DEPT_ID)
        REFERENCES DEPARTMENTS(DEPT_ID)
);

CREATE TABLE COURSES (
    COURSE_ID   NUMBER(10) PRIMARY KEY,
    COURSE_NAME VARCHAR2(100) NOT NULL,
    CREDITS     NUMBER(2),
    DEPT_ID     NUMBER(10),
    CONSTRAINT FK_COURSES_DEPT FOREIGN KEY (DEPT_ID)
        REFERENCES DEPARTMENTS(DEPT_ID)
);

CREATE TABLE INSTRUCTORS (
    INSTRUCTOR_ID NUMBER(10) PRIMARY KEY,
    FIRST_NAME    VARCHAR2(50) NOT NULL,
    LAST_NAME     VARCHAR2(50) NOT NULL,
    HIRE_DATE     DATE,
    DEPT_ID       NUMBER(10),
    CONSTRAINT FK_INSTRUCTORS_DEPT FOREIGN KEY (DEPT_ID)
        REFERENCES DEPARTMENTS(DEPT_ID)
);

CREATE TABLE ENROLLMENTS (
    ENROLLMENT_ID   NUMBER(10) PRIMARY KEY,
    STUDENT_ID      NUMBER(10) NOT NULL,
    COURSE_ID       NUMBER(10) NOT NULL,
    ENROLLMENT_DATE DATE,
    GRADE           VARCHAR2(2),  -- отсутствует в PROD
    CONSTRAINT FK_ENROLL_STUDENT FOREIGN KEY (STUDENT_ID)
        REFERENCES STUDENTS(STUDENT_ID),
    CONSTRAINT FK_ENROLL_COURSE FOREIGN KEY (COURSE_ID)
        REFERENCES COURSES(COURSE_ID)
);
```

3.3 Определение структурных различий

Для поиска различий между схемами используются операции *MINUS* и *LEFT JOIN* с системными представлениями. Функция определяет три типа различий:

Таблица 3.1 – Типы различий между схемами

Тип	Описание	Пример
MISSING	Объект есть в DEV, отсутствует в PROD	Таблица INSTRUCTORS
DIFFERENT	Структура объекта различается	Добавлена колонка EMAIL
EXTRA_IN_PROD	Объект есть в PROD, отсутствует в DEV	Устаревшая таблица

Определение таблиц, отсутствующих в Prod, выполняется с помощью операции *MINUS*:

Листинг 3.3 – Поиск отсутствующих таблиц в PROD

```
FOR r IN (
    SELECT table_name FROM all_tables WHERE owner =
UPPER(p_dev_schema)
    MINUS
    SELECT table_name FROM all_tables WHERE owner =
UPPER(p_prod_schema)
) LOOP
    v_tables_to_report(r.table_name) := 'Table missing in PROD';
END LOOP;
```

Определение различий в структуре колонок выполняется через *LEFT JOIN* с проверкой типа данных, длины и признака *NULL*:

Листинг 3.4 – Сравнение структуры колонок

```
FOR r IN (
    SELECT dev.table_name,
           'Structure mismatch: ' || LISTAGG(dev.column_name || ' ('
||
           dev.data_type || ')', ', ') WITHIN GROUP (ORDER BY
dev.column_id) as diff_info
    FROM all_tab_columns dev
    LEFT JOIN all_tab_columns prod
      ON dev.table_name = prod.table_name
     AND dev.column_name = prod.column_name
     AND prod.owner = UPPER(p_prod_schema)
    WHERE dev.owner = UPPER(p_dev_schema)
      AND (prod.column_name IS NULL
           OR dev.data_type <> prod.data_type
           OR dev.data_length <> prod.data_length
           OR dev.nullable <> prod.nullable)
    GROUP BY dev.table_name
) LOOP
    IF NOT v_tables_to_report.EXISTS(r.table_name) THEN
        v_tables_to_report(r.table_name) := r.diff_info;
```

```
    END IF;
END LOOP;
```

3.4 Топологическая сортировка с помощью DFS

Для определения порядка создания таблиц реализован алгоритм обхода в глубину. Рекурсивная процедура *visit_table* обходит зависимости через внешние ключи и определяет порядок обработки таблиц:

Листинг 3.5 – Рекурсивная процедура обхода зависимостей

```
PROCEDURE visit_table(p_tn IN VARCHAR2, p_stack_path IN VARCHAR2
DEFAULT '') IS
    v_current_path VARCHAR2(4000);
BEGIN
    IF v_has_cycle THEN RETURN; END IF;

    -- Обнаружение цикла (попали в серую вершину)
    IF v_all_dev_tables.EXISTS(p_tn) AND v_all_dev_tables(p_tn) = 1
THEN
        v_has_cycle := TRUE;
        v_cycle_msg := 'Circular dependency: ' || p_stack_path || ' ->
' || p_tn;
        RETURN;
    END IF;

    -- Вершина еще белая (не посещена)
    IF v_all_dev_tables.EXISTS(p_tn) AND v_all_dev_tables(p_tn) = 0
THEN
        v_all_dev_tables(p_tn) := 1;  -- Переводим в серую
        v_current_path := p_stack_path ||
            (CASE WHEN p_stack_path IS NULL THEN '' ELSE ' -> ' END)
|| p_tn;

        -- Обходим все таблицы, на которые ссылается данная
        FOR r IN (
            SELECT DISTINCT r.table_name as parent_table
            FROM all_constraints c
            JOIN all_constraints r ON c.r_constraint_name =
r.constraint_name
                AND c.r_owner = r.owner
            WHERE c.owner = UPPER(p_dev_schema)
              AND c.table_name = UPPER(p_tn)
              AND c.constraint_type = 'R'
              AND r.table_name <> c.table_name
        ) LOOP
            visit_table(r.parent_table, v_current_path);
        END LOOP;

        v_all_dev_tables(p_tn) := 2;  -- Переводим в черную
        v_sort_counter := v_sort_counter + 1;
        v_sorted_indices(p_tn) := v_sort_counter;
    END IF;
END visit_table;
```

3.5 Сравнение процедур, функций, индексов и пакетов

Для сравнения других объектов базы данных используются представления *ALL_OBJECTS* и *ALL_PROCEDURES*. Функция определяет объекты, отсутствующие в Prod или существующие только в Prod:

Листинг 3.6 – Сравнение процедур и функций

```
FOR r IN (
    SELECT object_name FROM all_objects
    WHERE owner = UPPER(p_dev_schema) AND object_type = 'PROCEDURE'
    MINUS
    SELECT object_name FROM all_objects
    WHERE owner = UPPER(p_prod_schema) AND object_type = 'PROCEDURE'
) LOOP
    -- Объект MISSING - нужен CREATE
END LOOP;
```

Для генерации DDL-скриптов используется *DBMS_METADATA.GET_DDL*, который возвращает полный текст объекта. Имя схемы из результата удаляется для возможности создания в Prod.

3.6 Генерация DDL-скриптов и удаление объектов

Задание 3 подразумевает не только поиск различий, но и генерацию готовых SQL-скриптов для синхронизации:

– *MISSING* – объект отсутствует в Prod, нужно создать: генерируется *CREATE TABLE*, *CREATE INDEX*, *CREATE OR REPLACE PROCEDURE/ FUNCTION/PACKAGE*

– *DIFFERENT* – структура объекта отличается, нужно изменить: генерируется *ALTER TABLE ADD COLUMN* для добавления отсутствующих колонок;

– *EXTRA_IN_PROD* – объект есть в Prod, но отсутствует в Dev, нужно удалить: генерируется *DROP TABLE*, *DROP INDEX* и т.д.

3.7 Результат выполнения

При вызове функции сравнения:

Листинг 3.7 – Вызов функции сравнения схем

```
SELECT obj_type, obj_name, status, details, sort_order
FROM TABLE(compare_schemas_oracle('DEV_USER', 'PROD_USER'))
ORDER BY sort_order;
```

Были обнаружены следующие различия:

	OBJ_NAME	STATUS	DETAILS	DDL_SCRIPT	SORT_ORDER
1	STUDENTS	DIFFERENT	Structure mismatch: EMAIL (VARCHAR2)	ALTER TABLE "STUDENTS" ADD "EMAIL" VARCHAR2(100);	3
2	ENROLLMENTS	DIFFERENT	Structure mismatch: GRADE (VARCHAR2)	ALTER TABLE "ENROLLMENTS" ADD "GRADE" VARCHAR2(2);	4
3	INSTRUCTORS	MISSING	Table missing in PROD schema	CREATE TABLE "INSTRUCTORS" (⏎ "INSTRUCTOR_ID" NUMBER(10…	5
4	IDX_STUDENTS_EMAIL	MISSING	Index missing in PROD schema	CREATE INDEX "IDX_STUDENTS_EMAIL" ON "STUDENTS" ("EMAIL"…	10005
5	UNIVERSITY_PKG	MISSING	Package missing in PROD schema	⏎ CREATE OR REPLACE PACKAGE "UNIVERSITY_PKG" AS⏎ -- …	20007
6	GET_STUDENT_COUNT	MISSING	Procedure missing in PROD schema	⏎ CREATE OR REPLACE PROCEDURE "GET_STUDENT_COUNT" (p_de…	30008
7	GET_STUDENT_EMAIL	MISSING	Function missing in PROD schema	⏎ CREATE OR REPLACE FUNCTION "GET_STUDENT_EMAIL" (p_stu…	40009
8	OLD_TABLE	EXTRA_IN_PROD	Table exists in PROD but not in DEV	DROP TABLE "OLD_TABLE" CASCADE CONSTRAINTS;	100000
9	IDX_OLD	EXTRA_IN_PROD	Index exists in PROD but not in DEV	DROP INDEX "IDX_OLD";	110000

Рисунок 3.1 – Результат сравнения схем DEV_USER и PROD_USER

Таблицы отсортированы с учётом зависимостей через *FOREIGN KEY*: сначала *STUDENTS* (зависит от *DEPARTMENTS*), затем *ENROLLMENTS* (зависит от *STUDENTS* и *COURSES*), затем *INSTRUCTORS*. Объекты с типом *EXTRA_IN_PROD* выводятся в конце для удаления.

Для синхронизации PROD с DEV необходимо выполнить DDL-скрипты для объектов со статусом *MISSING* и *DIFFERENT*, а затем удалить объекты со статусом *EXTRA_IN_PROD*.

ВЫВОД

В ходе выполнения лабораторной работы была разработана функция для сравнения двух схем Oracle. Были изучены и применены на практике:

– Системные представления Oracle (*ALL_TABLES*, *ALL_TAB_COLUMNS*, *ALL_CONSTRAINTS*) для получения метаданных базы данных;

– Операции множеств (*MINUS*) для определения отсутствующих объектов;

– Алгоритм топологической сортировки на основе обхода в глубину для определения порядка создания таблиц с учётом внешних ключей;

– Pipelined-функции для эффективного возврата структурированных данных.

Разработанная функция позволяет автоматически определять различия между схемами и рекомендовать порядок создания/обновления объектов, что существенно упрощает процесс синхронизации баз данных между окружениями разработки и промышленной эксплуатации. Обнаружение циклических зависимостей предотвращает некорректное развертывание изменений, а учёт объектов, отсутствующих в схеме разработки, позволяет идентифицировать устаревшие элементы структуры.